CO1-ASSESSMENT TOOL 3

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: DSA0311

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Aim: To validate an email address, password, and mobile number using Python regular expressions.

CODE:

```
import re
email = input("Enter email: ")
password = input("Enter password: ")
mobile = input("Enter mobile number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

OUTPUT:

```
Enter email: student123@saveetha.edu
Enter password: Student@123
Enter mobile number: 9876543210
Valid Email
Strong Password
Valid Mobile Number
```

RESULT: The email, password, and mobile number validator was successfully implemented using Python Regular Expressions. It displays valid or invalid messages based on the given rules.

Section 2: Design a Finite State Automata (FSA) Simulator

1. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Aim: To implement a Deterministic Finite Automaton (DFA) that accepts strings ending with ab.

CODE:

```
states = input("Enter states: ").split()
alphabet = input("Enter input alphabet: ").split()
transition = {}
for state in states:
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(
            f"Transition for {state} with {symbol}: "
        )
        transition[state][symbol] = next_state
initial = input("Enter initial state: ")
final_states = input("Enter final states: ").split()
n = int(input("Enter number of strings: "))
for i in range(n):
```

```
    string = input("Enter string: ")
    current = initial
    path = [current]
    valid = True
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        current = transition[current][symbol]
        path.append(current)
    print("Transition Path:", " -> ".join(path))
    if valid and current in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

OUTPUT:

```
Enter states: q0 q1 q2
Enter input alphabet:  a b
Transition for q0 with a: q1
Transition for q0 with b: q0
Transition for q1 with a:  q1
Transition for q1 with b:  q2
Transition for q2 with a: q1
Transition for q2 with b: q0
Enter initial state:  q0
Enter final states: q2
Enter number of strings: 2
Enter string: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string:  aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Rejected
```

RESULT: The DFA simulator was successfully implemented in Python. It displays the transition path and determines whether the input strings are accepted or rejected.

Section 3: Smart Pattern Matching Engine using Regular Expressions

1. **Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Aim: To search text for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions using Python regular expressions.

CODE:

```
import re

text = input("Enter text: ")

print("\n1. Search Word")

print("2. Search Date")

print("3. Search Phone Number")

print("4. Search Hashtag")

print("5. Search Mention")

print("6. Search Prefix")

print("7. Search Suffix")

choice = input("Enter your choice: ")

if choice == "1":

    word = input("Enter word: ")

    pattern = r'\b' + re.escape(word) + r'\b'

    result = re.findall(pattern, text, re.I)

elif choice == "2":

    pattern = r'\b\d{2}/\d{2}/\d{4}\b'

    result = re.findall(pattern, text)

elif choice == "3":
```

```
    pattern = r'\b[6-9]\d{9}\b'
    result = re.findall(pattern, text)
elif choice == "4":
    pattern = r'(?<!\w)#[A-Za-z0-9_]+'
    result = re.findall(pattern, text)
elif choice == "5":
    pattern = r'(?<!\w)@[A-Za-z0-9_]+'
    result = re.findall(pattern, text)
elif choice == "6":
    word = input("Enter prefix: ")
    pattern = r'\b' + re.escape(word) + r'\w*'
    result = re.findall(pattern, text, re.I)
elif choice == "7":
    word = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(word) + r'\b'
    result = re.findall(pattern, text, re.I)
else:
    result = None
    print("Invalid choice")
if result is not None:
    if result:
        print("\nMatching Patterns:")
        for item in result:
            print(item)
        print("Total Matches:", len(result))
    else:
        print("No matching patterns found")
```

OUTPUT:

```
Enter text: Meeting on 12/09/2026. Call 9876543210 #NLP @OpenAI natural language processing

1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
Enter your choice: 2

Matching Patterns:
12/09/2026
Total Matches: 1
```

RESULT: The pattern matching engine was successfully implemented using Python Regular Expressions. It searches for words, dates, phone numbers, hashtags, mentions, prefixes, and suffixes and displays the matching results.